

Offline Inventory Management System

Technical & Development Plan — Windows Desktop Application

Recommended stack: Python 3.13+ · PySide6 · SQLAlchemy · SQLite · PyInstaller

Purpose: define a practical, maintainable plan for a Windows inventory application that operates without an internet connection and stores its operational data locally.

Version: 1.0

Date: September 2026

1. Executive Summary

The proposed system is a native-style Windows desktop inventory application built with Python. The application will run fully offline, with SQLite as the local database. The user interface will be developed with PySide6 (Qt), business logic will be separated from the UI, and SQLAlchemy will provide structured database access. PyInstaller will package the application for Windows deployment.

Recommended technology stack

Area	Technology	Purpose
Programming	Python 3.13+	Application development
Desktop UI	PySide6 / Qt	Windows forms, tables, dialogs and navigation
Database	SQLite	Local, serverless data storage
ORM	SQLAlchemy	Database models, queries and transactions
Reports	ReportLab / openpyxl	PDF reports and Excel export
Packaging	PyInstaller	Windows executable/package
Testing	pytest	Automated unit and integration tests

2. Objectives

- Operate completely without internet access.
- Maintain accurate product and stock records in a local database.
- Record every stock movement with a traceable transaction history.
- Provide fast product search, filtering and stock-status visibility.
- Support purchases/stock-in, stock-out/sales, stock adjustments and reporting.
- Provide local backup and restore facilities to reduce the risk of data loss.
- Keep the codebase modular so the system can later support barcode scanning or LAN-based multi-user operation.

3. Scope of the First Release

Core modules

- Dashboard: product count, stock value, low-stock items, out-of-stock items and recent transactions.
- Product Master: SKU, barcode, name, category, unit, prices, stock limits, location and status.
- Category / Unit / Location Masters: controlled reference data for products.
- Supplier Master: supplier contacts, address and tax information where applicable.
- Customer Master: customer information if stock-out/sales transactions require it.
- Purchase / Stock In: supplier, invoice/reference, line items, quantities, prices and totals.
- Stock Out / Sale: customer/reference, line items, quantities and prices.
- Stock Adjustment: physical-vs-system reconciliation with a mandatory reason.
- Stock Ledger: complete movement history by product.
- Reports: current stock, low stock, stock movement, purchase/sales and valuation reports.
- Users & Permissions: administrator, manager, operator and viewer roles.
- Backup & Restore: manual backup plus configurable automatic local backups.

4. High-Level Architecture

UI Layer → Application Services → Domain Models → Data Access → SQLite

PySide6 will handle the Windows interface. ViewModels/controllers will coordinate user actions.

Application services will enforce business rules. SQLAlchemy will handle database persistence. SQLite will remain local to the machine.

Logical flow

User → PySide6 Screen → ViewModel/Controller → Service → SQLAlchemy → SQLite database → Result → UI

5. Database Design

Table	Purpose
users	Application users and password hashes
roles / permissions	Access control
products	Product master and current stock metadata
categories	Product categories
units	Measurement units
locations	Warehouse/rack/bin locations
suppliers	Supplier master
customers	Customer master
purchases	Purchase/stock-in headers
purchase_items	Purchase line items
sales	Stock-out/sales headers
sale_items	Stock-out/sales line items
stock_transactions	Immutable stock movement ledger
stock_adjustments	Physical stock reconciliation
audit_logs	Important user/system actions
application_settings	Local configuration

Stock ledger principle

Do not make the current stock quantity the only source of truth. Every receipt, issue, sale, damage, return or adjustment should create a stock transaction. The application may maintain a current-stock value for fast display, but the ledger must remain available for audit and reconstruction.

Example stock movement

Opening 100 → Purchase +50 → Sale -20 → Damage -2 → Balance 128.

6. Main Screens

- Login
- Dashboard
- Products list / add / edit / view
- Categories, units and locations
- Suppliers

- Customers
- Purchase / Stock In
- Stock Out / Sale
- Stock Adjustment
- Stock Ledger / Product History
- Reports
- User Management
- Backup / Restore
- Application Settings

7. Key Business Rules

- SKU/item code must be unique.
- Products should not be hard-deleted once they have historical transactions; use inactive status instead.
- Stock-affecting operations must run inside database transactions.
- Every stock change must have a source/reference and timestamp.
- Negative stock should be blocked by default, with a configurable policy if the business requires it.
- Stock adjustment requires a reason and should be recorded in the ledger.
- Prices and quantities must use appropriate numeric/decimal handling rather than binary floating-point arithmetic for financial values.
- Important actions such as deletion, adjustment, restore and user changes should be captured in audit logs.

8. Offline Storage & Backup

The operational database should be stored outside the installation directory, for example under C:\ProgramData\YourCompany\Inventory\. The application executable can be installed under Program Files. This separation simplifies upgrades and permissions.

Backup strategy

- Daily automatic local backup.
- Keep a configurable number of daily/weekly backup generations.
- Manual “Backup Now” option.
- Restore workflow with confirmation and validation.
- Allow backup destination to be changed to a USB drive, external disk or LAN folder.
- Provide database integrity checks after restore.
- Document that local-only backup does not protect against physical loss unless backups are copied to another device.

9. Security

- Use salted password hashes; never store plain-text passwords.
- Role-based permissions for sensitive functions.
- Lock or require re-authentication for administrative actions where appropriate.
- Audit important data-changing operations.

- Restrict database file access using normal Windows filesystem permissions.
- Protect restore and database maintenance functions from ordinary operators.

10. Reporting & Export

- Current stock report.
- Low-stock and out-of-stock report.
- Product transaction history.
- Stock movement report by date range.
- Purchase report by supplier/date.
- Stock-out/sales report by customer/date.
- Inventory valuation report.
- PDF report generation using ReportLab.
- Excel/CSV export using openpyxl/pandas where appropriate.

11. Suggested Project Structure

```
InventoryManagement/
├── app/
│   ├── ui/ # PySide6 windows, dialogs and widgets
│   ├── viewmodels/ # UI state and commands
│   ├── services/ # Business operations
│   ├── domain/ # Entities and business rules
│   ├── data/
│   │   ├── models/ # SQLAlchemy models
│   │   ├── repositories/ # Database access
│   │   └── migrations/ # Schema migrations
│   ├── reports/ # PDF/Excel reporting
│   ├── security/ # Authentication/permissions
│   ├── backup/ # Backup/restore
│   └── config/ # Application settings
├── tests/
├── resources/
├── main.py
├── requirements.txt
└── build/
```

12. Development Roadmap

Phase	Deliverables
1. Foundation	Project setup, PySide6 shell, configuration, logging, SQLite, SQLAlchemy, migrations
2. Master Data	Products, categories, units, locations, suppliers and customers
3. Inventory	Opening stock, stock-in, stock-out, adjustments and stock ledger
4. Dashboard & Reports	KPIs, searches, filters, PDF/Excel/CSV reports
5. Security	Login, roles, permissions and audit log
6. Backup & Deployment	Automatic/manual backup, restore, installer and Windows packaging
7. Testing	Unit, integration, UI, backup/restore and failure-recovery testing

13. Testing Strategy

- Unit tests for stock calculations, validation, permissions and business rules.
- Integration tests for database transactions and migrations.

- UI tests for critical workflows such as purchase, stock-out and adjustment.
- Boundary tests for zero stock, large quantities, duplicate SKUs and invalid data.
- Backup/restore tests, including interrupted operations and corrupted backup files.
- Power-loss/failure testing around stock transactions to verify database consistency.
- Performance tests with a realistic product and transaction volume.

14. Packaging & Deployment

- Develop and test on a supported Windows version.
- Build a self-contained Windows package with PyInstaller.
- Use a proper installer for shortcuts, application files and uninstall support.
- Keep user data/database outside the application installation directory.
- Provide a first-run setup flow for administrator creation and backup location.
- Version database schema using migrations so future releases can upgrade existing installations safely.

15. Future Enhancements

- USB barcode scanner support.
- Barcode/label printing.
- Purchase orders and sales invoices.
- Returns and cancellations.
- Batch/lot and expiry tracking.
- Multiple warehouses.
- LAN multi-user mode.
- Role-specific dashboards.
- Advanced inventory valuation methods.
- Optional future synchronization/cloud module, if business requirements change.

16. Recommended MVP Definition

The first production-ready version should focus on reliable stock control rather than trying to include every possible business feature. The MVP should include: product master, categories/units/locations, suppliers, opening stock, stock-in, stock-out, stock adjustments, stock ledger, dashboard, essential reports, login/roles, and backup/restore.

Recommended implementation decision

Choose Python + PySide6 + SQLAlchemy + SQLite if the application is primarily for one Windows PC or a small number of users and you value rapid development and flexibility. If several PCs must write to one shared inventory concurrently, plan a LAN architecture with a database server/API instead of placing a writable SQLite file on a shared network drive.

17. Conclusion

This architecture provides a practical offline Windows inventory system with local ownership of data, clear separation between interface and business logic, traceable stock movements, and a straightforward path to future features. The most important engineering priorities are transaction integrity, backup/restore, auditability and a clean separation of application layers.